Advanced Computer Systems

Lecture 1

Dr. Shounak Chakraborty,
Assistant Professor,
Dept. of Computer Science,
Durham University, UK

Oct 6, 2026

Durham University

COMP3791

Course Metadata

- **Prerequisites:**
  - **COMP1071 Computer Systems AND**
  - **COMP2221 Programming Paradigms AND**
  - **COMP1021 Maths for Computer Science OR**
    - **MATH1551 Maths for Engineers and Scientists OR**
    - **MATH1561 Single Mathematics A AND MATH1571 Single Mathematics B OR**
    - **MATH1061 Calculus I AND MATH1017 Linear Algebra I**

- **Course Outline:**
  - **Characteristics and design of current processor architectures**
  - **Memory systems: cache design and performance**
  - **Compiler frontend: Foundations of compiler theory and techniques for lexical analysis, syntax analysis and semantic analysis**
  - **Compiler backend: Intermediate representations (IRs), optimisation on IRs, machine code generation and register allocation**

Module Information (Advanced Computer Architecture)

- **Schedule/Time-Table:**
  - **Weeks: 12, 14, 16, 18, 20 (6 Oct 2026 to 1 Dec 2026) [Lectures]**
  - **Weeks: 13, 15, 17, 19, 21 (13 Oct 2026 to 8 Dec 2026) [Workshops]**
  - **Tuesday: 9:00 - 11:00 hrs (E102)**

- **Instructor: Dr. Shounak Chakraborty**
  - **Office Location & Timings: *MCS2104* (Tuesday: 16:00 - 17:00 hrs)**
  - **Please drop me an email in case you want to meet outside of these hours**
  - **Reach me: *shounak.chakraborty@durham.ac.uk***

- **Course Materials-**
  - **Computer Architecture: A Quantitative Approach (The Morgan Kaufmann Series in Computer Architecture and Design) Paperback – 15 December 2017 by John L. Hennessy and David A. Patterson**
  - **Parallel Computer Organization and Design (Cambridge University Press) [Second Edition] by Michael Dubois, Murali Annavaram, and Per Stenstrom**
  - **Lecture Slides (and/or Handouts), Relevant Research papers [will be discussed during the lectures]**
  - **Simulators/Tools (maybe for the Workshop-sessions)**

- **Assessments: Assignment and Exam**

Course Outline (Adv. Comp. Arch.)

- **Introduction and Processor Fundamentals (Lec 5 hrs; WS 5 hrs)**
  - **Dennard Scaling, The Energy/Thermal Wall**
  - **ILP, Branches & BTB, Out-of-Order Execution**
  - **Data Parallelism, Vectors, Threads & Multi-cores**
  - **Load-Store Queues (LSQ), memory dependence speculation, store forwarding**
- **Caches, Memory and Optimisations (Lec. 5 hrs; WS 5 hrs)**
  - **Memory Hierarchies, Cache Designs, Write Policies**
  - **Cache Optimisations and 4Cs**

Microarchitectural Constraints

- **Historical evolution of processor metrics, the sudden breakdown of Dennard scaling, and the physical limits defining modern energy-constrained computing**

The Classical Scaling Era

- **From the mid-1970s until 2004, microarchitecture rode the predictable wave of transistor miniaturisation, delivering annual clock frequency increases without prohibitive power penalties**



- **Processor clock frequencies expanded ~50% annually until 2004, abruptly saturating around ~3.8 GHz due to thermal limits (e.g. Pentium 4)**

The Performance Triad

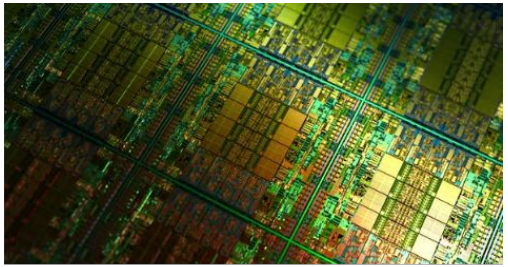
- **Iron Law of Processor Performance**
  - Execution Time = ***Instructions/Program * CPI * 1/f***
  - Performance is a three-way balancing act:
    - **Algorithm & Compiler:** Influence dynamic instruction count
    - **Microarchitecture:** Determines average CPI or IPC
    - **Circuit Design & Process Technology:** Set the maximum attainable clock frequency (f)

- **Latency vs. Throughput**
  - **Latency:** Time required to complete a single task or instruction from start to finish (critical for single-thread responsiveness)
  - **Throughput:** Total amount of work completed per unit time across all execution units (e.g., aggregate IPC, FLOPs/sec)
  - Deep pipelining improves throughput by overlapping operations, but often increases individual instruction latency due to hazard penalties and pipeline register overheads

- **The Third Dimension: Power & Energy Limits**
  - Faster clock frequencies and wider execution engines drive up power consumption:
    $P_{total} = P_{dynamic} + P_{static} = \alpha.C.V^2.f + I_{leak}.V$
  - Performance gains cannot be evaluated in isolation from **Thermal Design Power (TDP)** and energy cost per instruction
  - Modern metric shifts: Optimization targets transitioned from raw execution speed to energy-delay metrics (e.g., Energy-Delay Product, ***EDP = Energy * Delay***

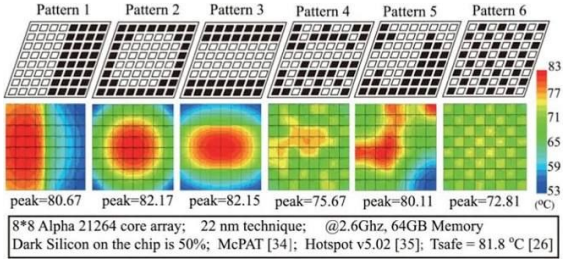
Dennard Scaling

- **Classical Promise (1974):** Shrinking MOSFET dimensions by factor *s* while reducing supply voltage (V_dd) by *1/s* kept chip power density constant (P / Area ≈ 1)
- **The "Free Lunch" Era:** Smaller transistors allowed clock frequencies to scale by *s* and densities by s^2 without increasing operating temperatures
- **The Voltage Wall (~2005):** Threshold voltage (V_th) hit subthreshold leakage limits; halting V_dd scaling around 1V to preserve noise margins
- **The Power Wall:** Transistor counts kept growing via Moore's Law, but static power leakage and constant V_dd caused power density to skyrocket
- **Architectural Shift:** Forced the end of frequency scaling (3 - 4 GHz ceiling), driving the transition to multi-core architectures, dark silicon constraints, and domain-specific accelerators



Dark Silicon

- **Definition:** The percentage of an integrated circuit's silicon area that cannot be powered on simultaneously at nominal frequency without exceeding the chip's TDP limit
- **The Root Cause:** Transistor counts continued scaling with Moore’s Law, but the stall in supply voltage (V_dd) scaling due to subthreshold leakage caused chip power density to escalate rapidly
- **The Utilisation Wall:** At deep nanometer nodes, as much as 50% to 80%+ of on-chip transistors must remain switched off ("dark") or heavily throttled ("dim") during peak execution
- **Failure of Homogeneous Multi-Core:** Adding more identical, general-purpose cores hits severe thermal throttling, yielding diminishing returns on parallel speedup (Amdahl's Law meets the Power Wall)
- **Architectural Remedy (Specialisation):** Spending silicon area on heterogeneous cores (e.g., big.LITTLE) and domain-specific accelerators (NPUs, DSPs) that execute with 10×–100× better energy efficiency and sleep when idle
- **Dynamic Management & Thermal Limits:** Relies on aggressive power gating, dynamic voltage and frequency scaling (DVFS), and thermally-aware scheduling to prevent localised hotspots and thermal runaway

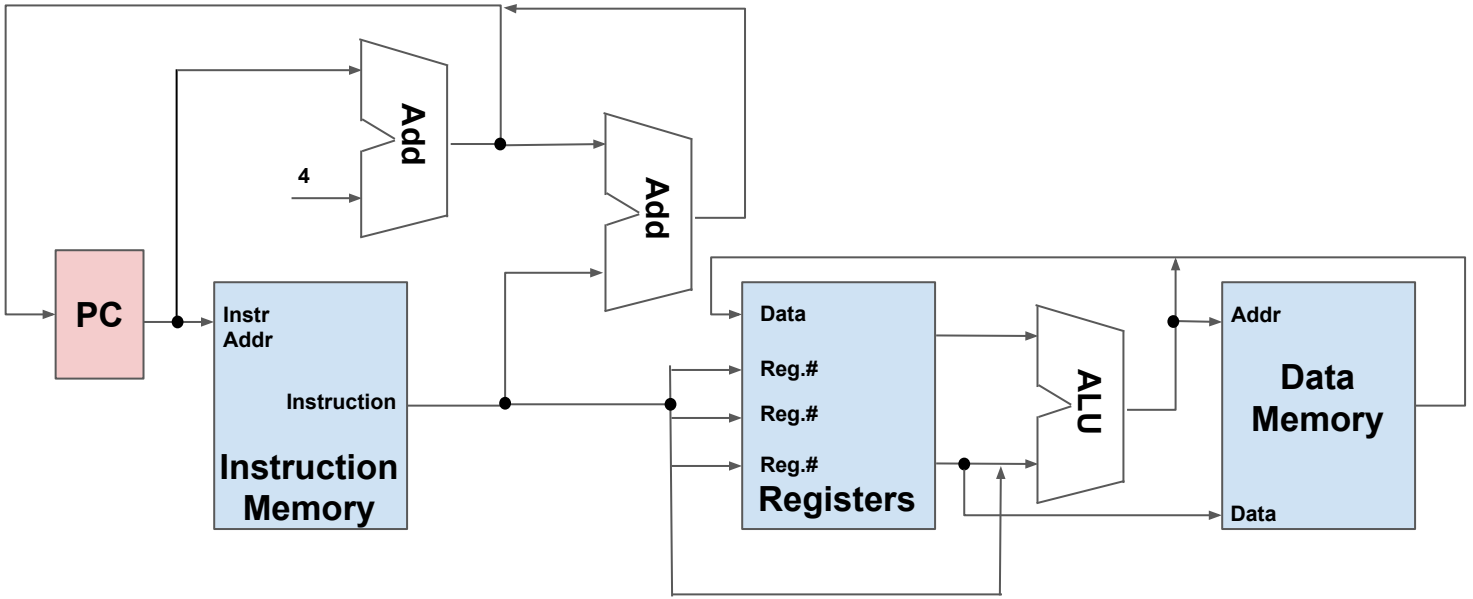


Architectural Consequences

- **Extinction of Mega-Pipelining:** The pursuit of 30+ pipeline stages (e.g., NetBurst) ceased immediately due to excessive latch power overhead, clock distribution dissipation, and severe branch misprediction penalties
- **The Multi-Core Revolution:** Single-thread frequency gains stalled; architects pivoted to chip-multiprocessors (CMP), relying on thread-level parallelism (TLP) and shared memory hierarchies to utilize transistor budgets
- **EDP Metric:** Performance-at-any-cost was replaced by MIPS/Watt, Joules per instruction, and aggressive microarchitectural clock-gating and power-gating techniques
- **Heterogeneous & Domain-Specific Silicon:** Modern chips feature asymmetric core complexes (e.g., big.LITTLE), integrated vector units (AVX/SVE), neural engines, and hardware accelerators to bypass the utilization wall.

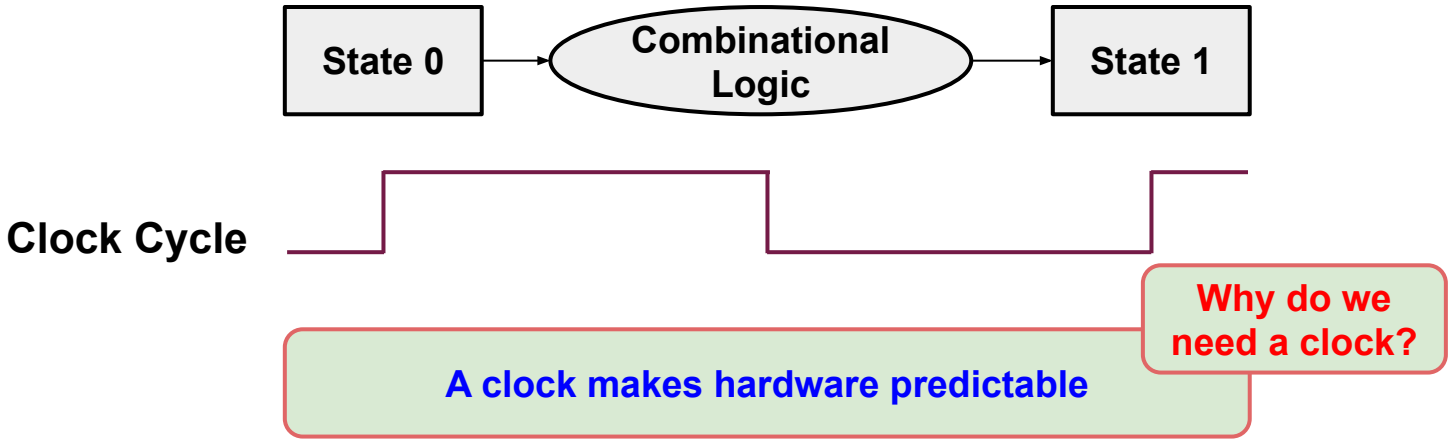
Processor & Pipeline Fundamentals

Basic MIPS Processor Design

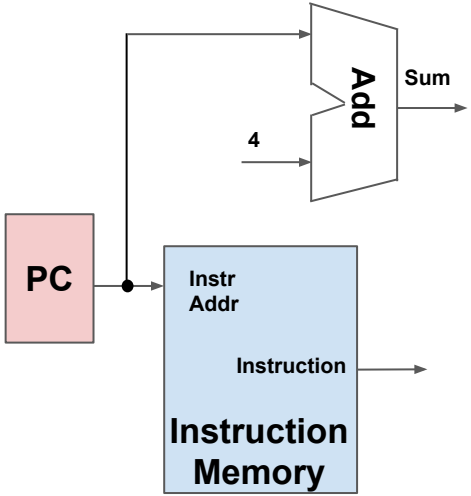


Logic Design Convention

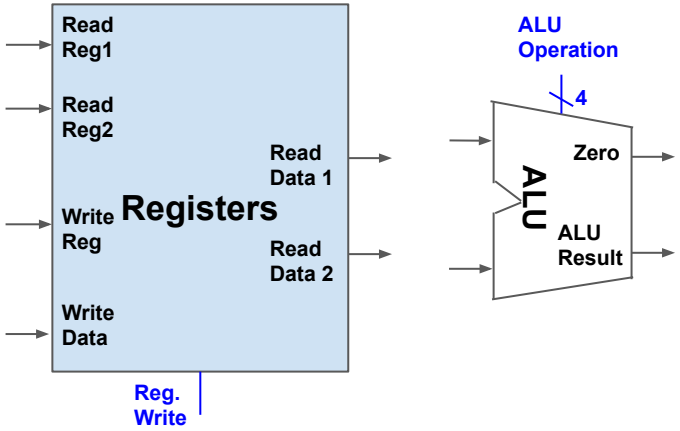
- Combinational vs Sequential
  - Which one is needed to build up an ALU and Why?
- Control Signals
  - Signals used for multiplexor selection or for directing the operation of the functional units
- Data Signals
  - contain information that is operated on by a functional unit



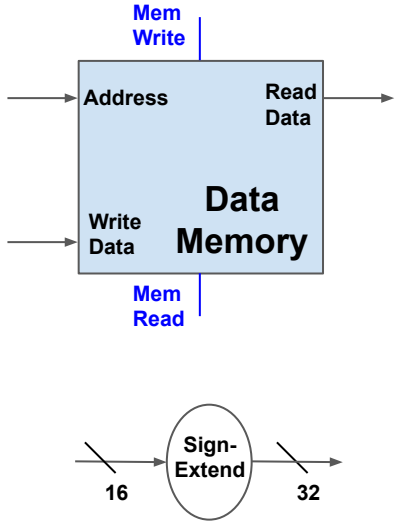
Building a Single Cycle MIPS Datapath: Components?



Instruction Memory, Fetch and PC

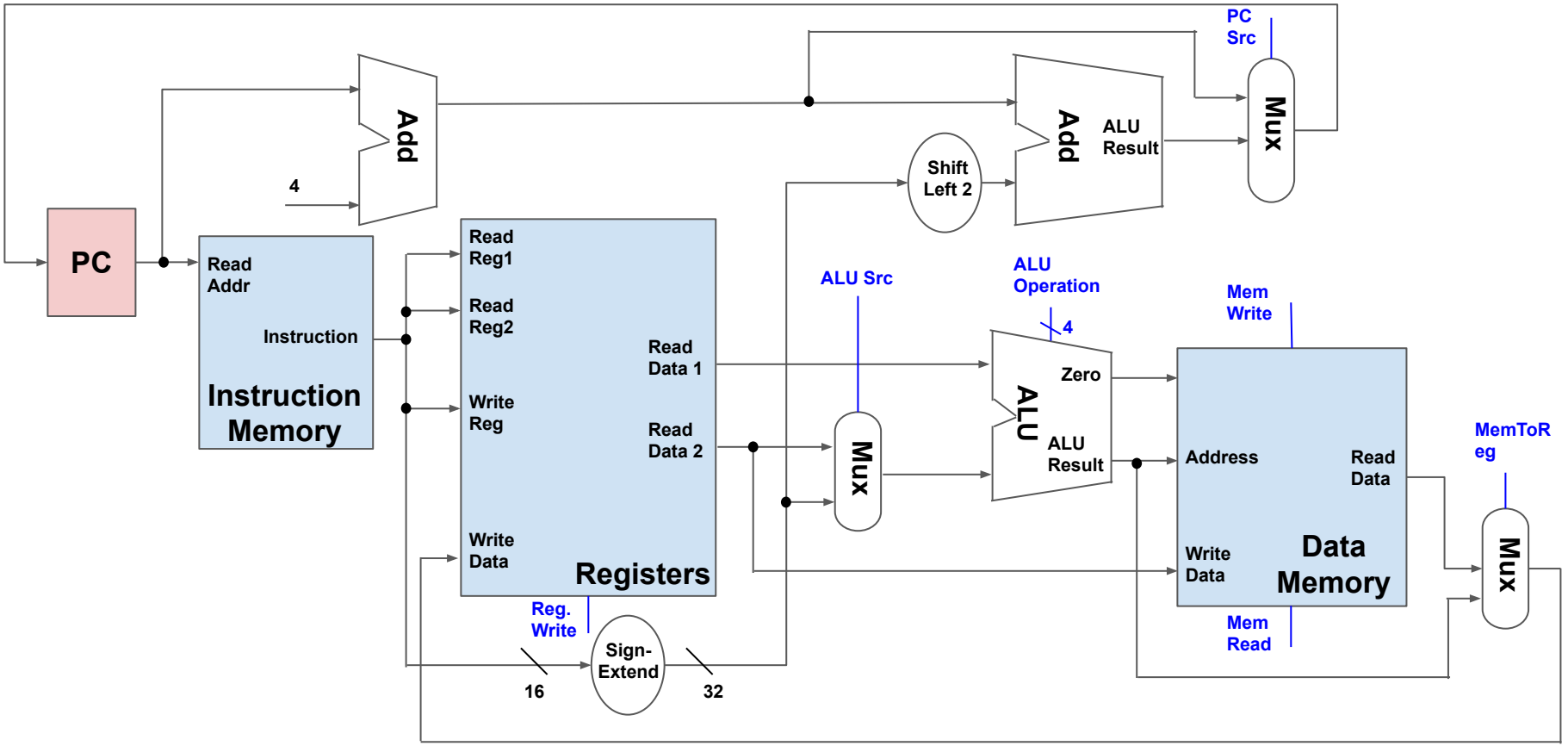


Registers and ALU



Load and Store

Building a Single Cycle MIPS Datapath



The ALU Control

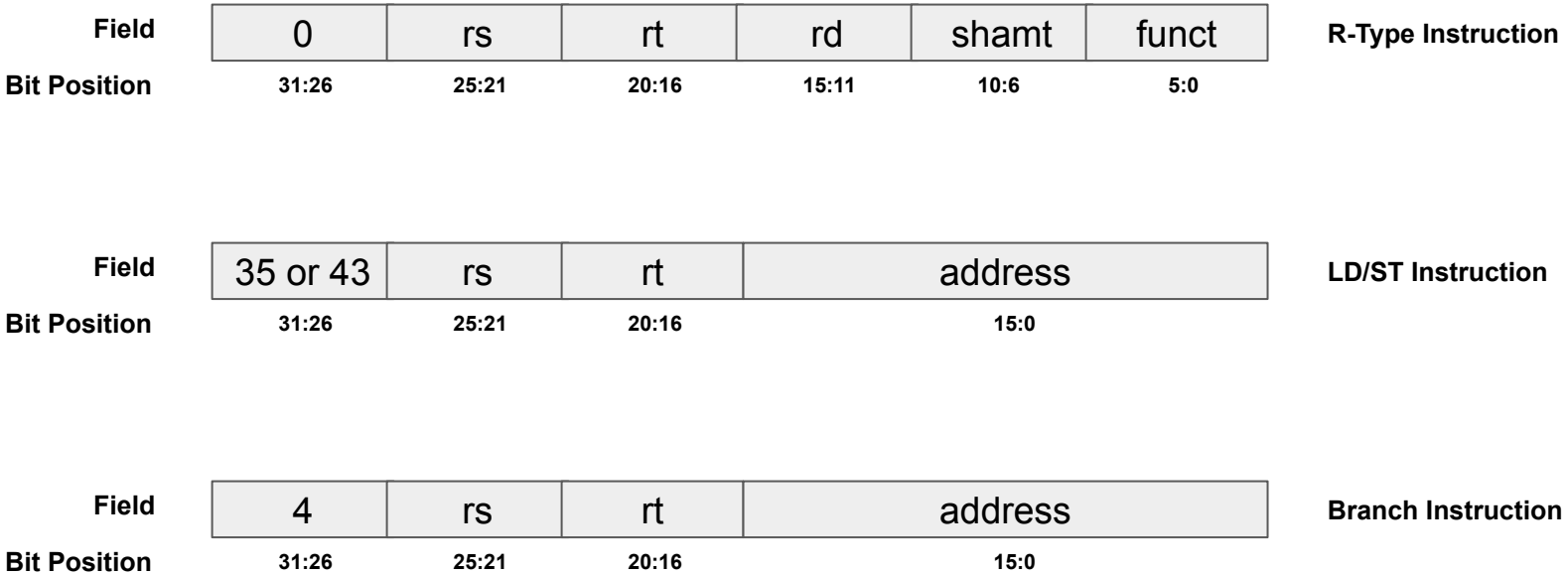
ALU Control Lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

MIPS ALU: Control Inputs

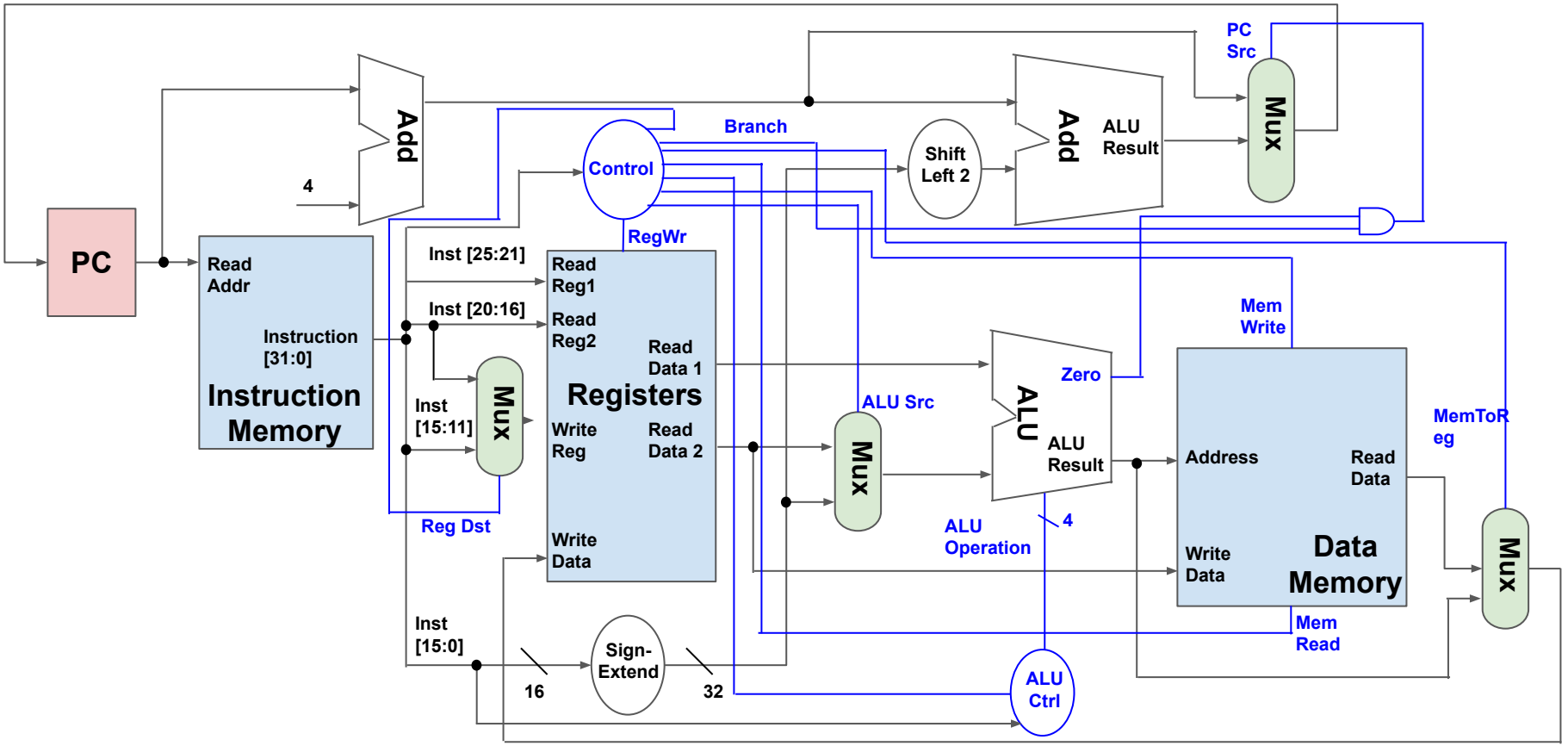
Instruction OpCode	ALUOp	Instruction Operation	Funct Field	Desired ALU Action	ALU Control Input
LW	00	Load word	XXXXXX	add	0010
SW	00	Store word	XXXXXX	add	0010
Branch EQ	01	Branch equal	XXXXXX	subtract	0110
R-Type	10	add	100000	add	0010
R-Type	10	subtract	100010	subtract	0110
R-Type	10	AND	100100	AND	0000
R-Type	10	OR	100101	OR	0001
R-Type	10	Set on less than	101010	Set on less than	0111

ALU control bits are set based on ALUOp control bits and different function codes for R-Type instructions

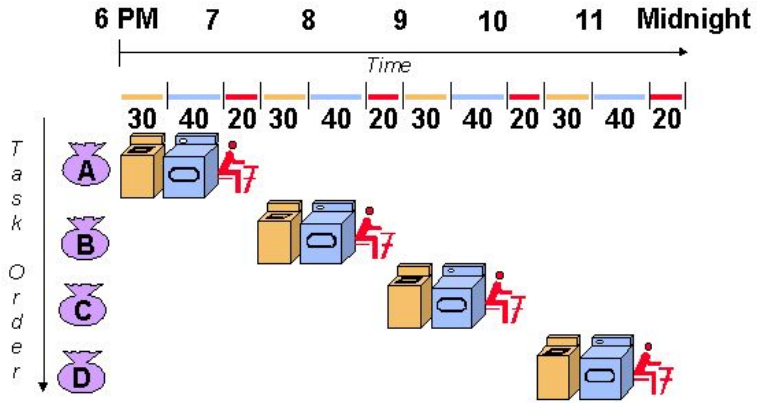
Instruction Classes

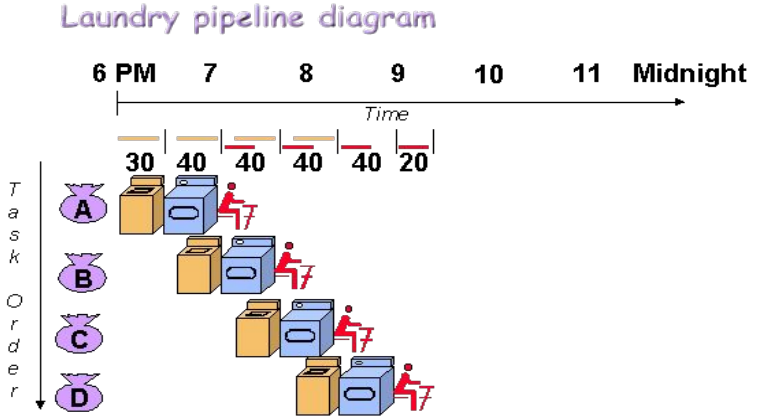


Single Cycle MIPS Datapath with Control Signals



Pipeline Overview





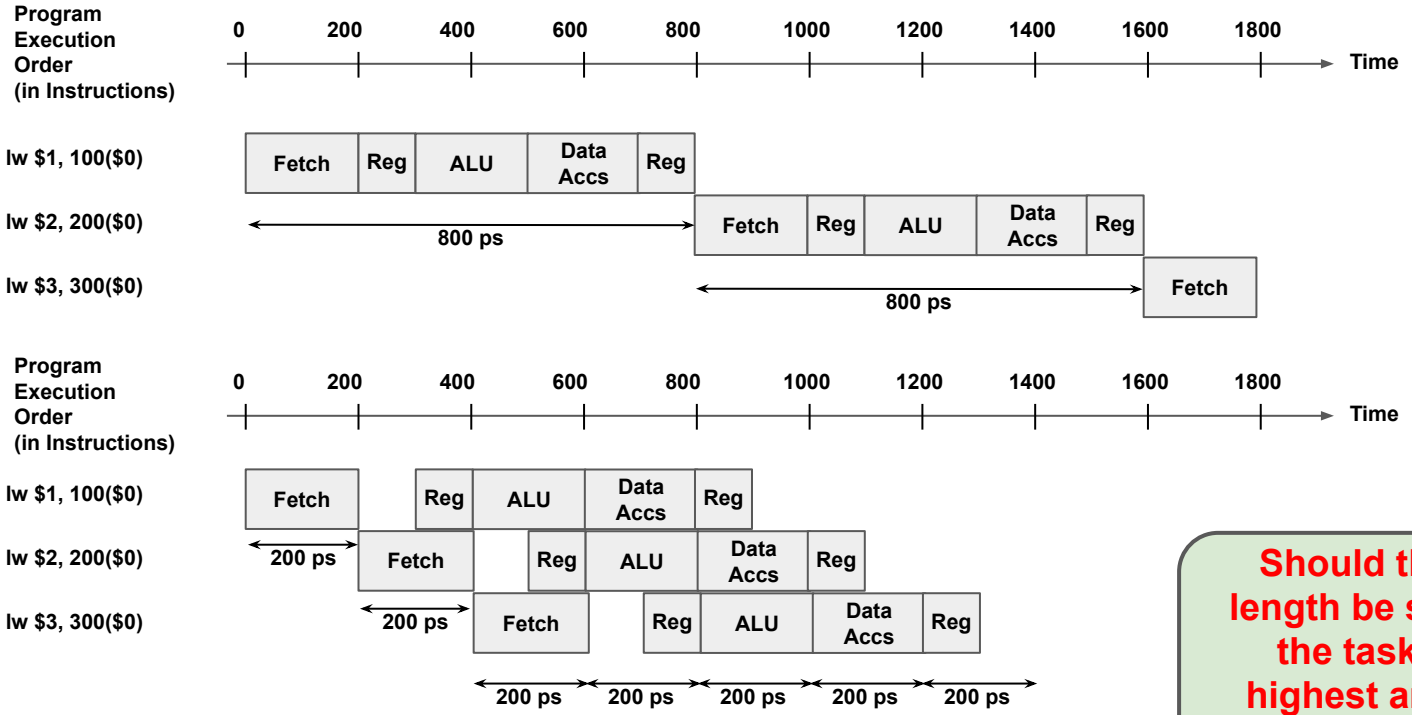
Laundry Example

Single Cycle vs. Pipelined Performance

- **Time_between_instructions_pipelined = (Time_between_instruction_nonpipelined) / (Number_of_pipeline_stages)**

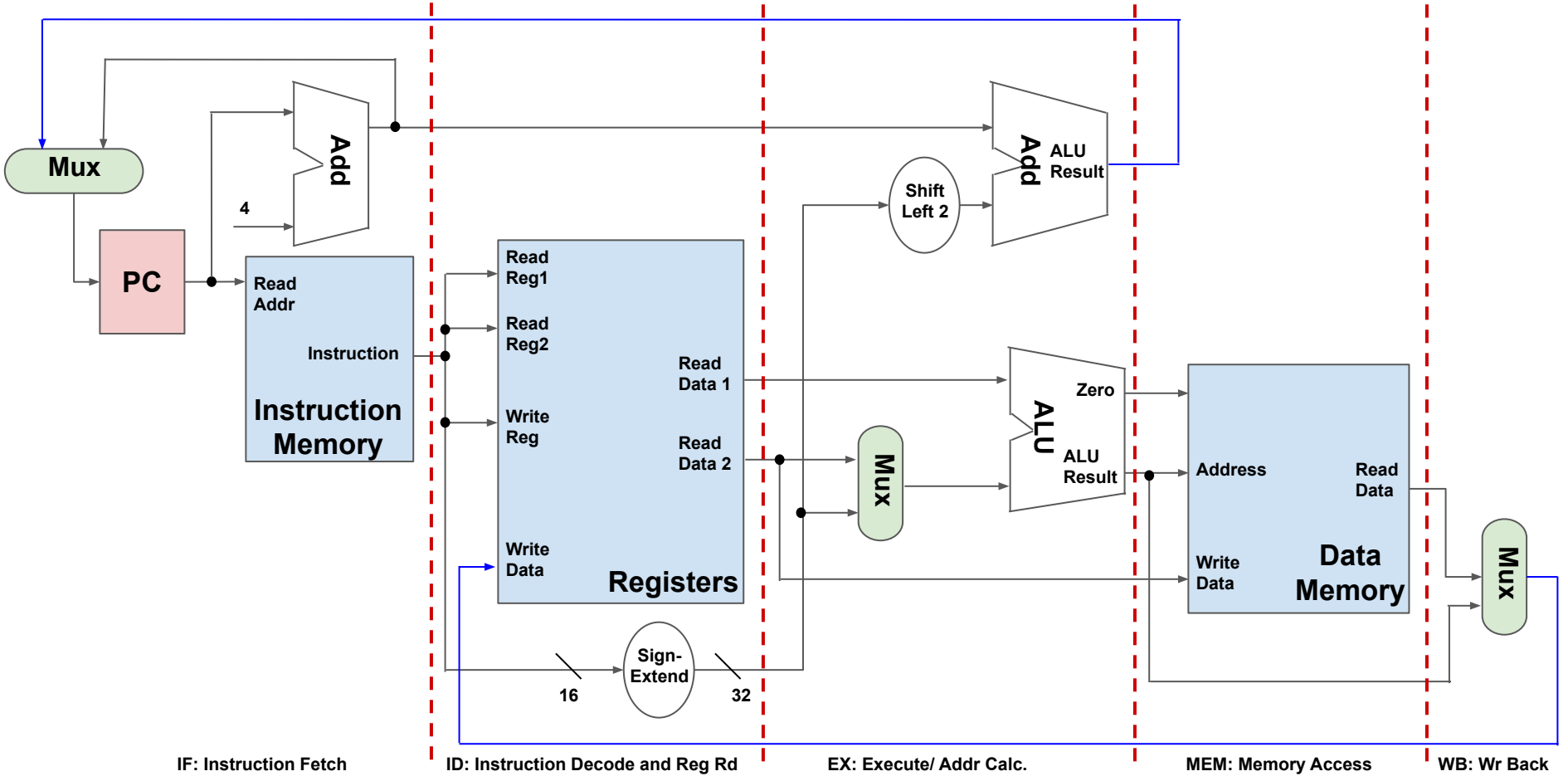
Instruction Class	Instruction Fetch	Register Read	ALU Operation	Data Access	Register Write	Total Time
Load Word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store Word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, AND, OR)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

Single Cycle vs. Pipelined Performance (cont..)

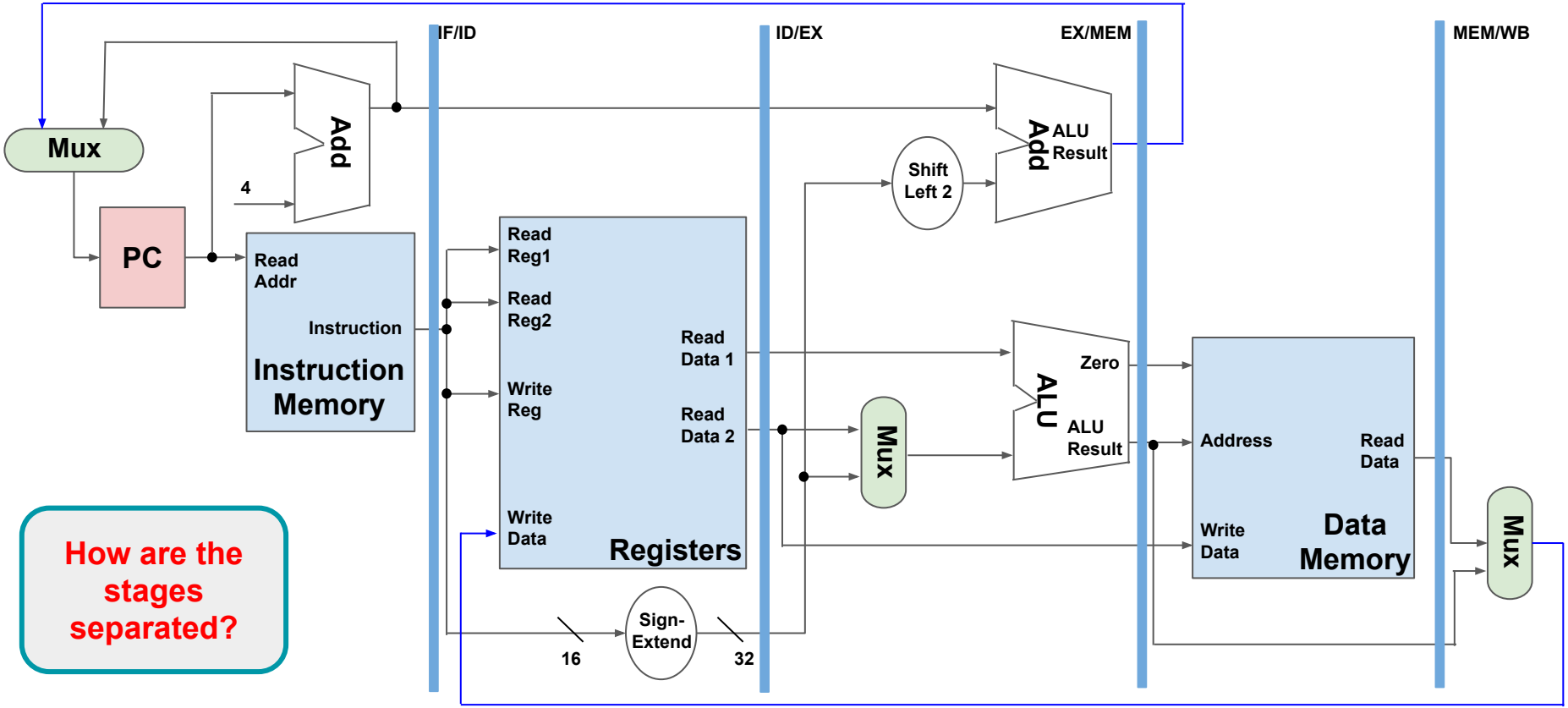


Should the cycle length be same with the task taking highest amount of time?

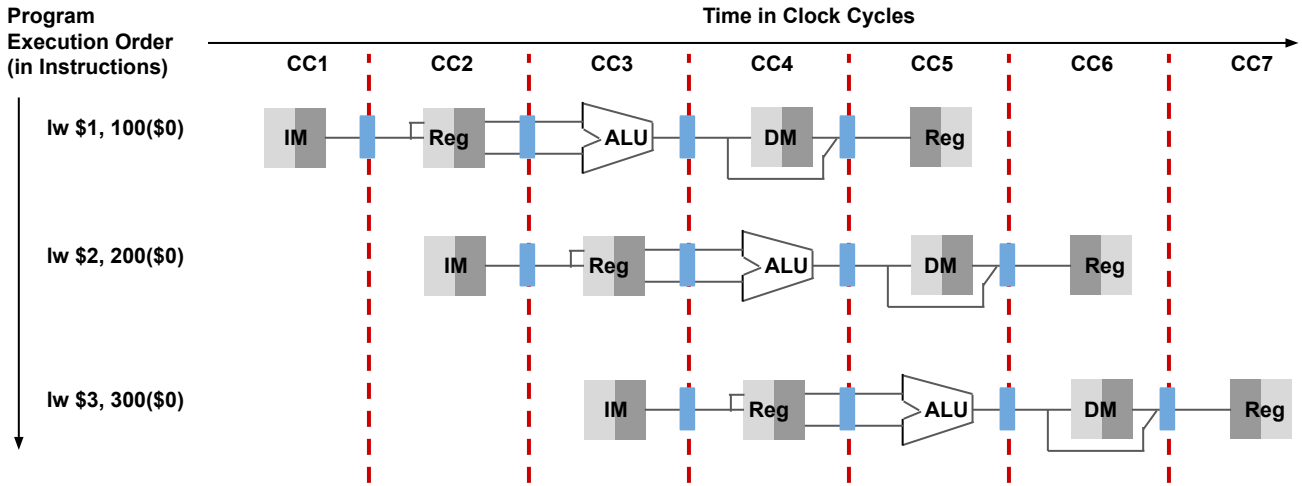
Pipeline Datapath and Control



Pipeline: State Transition



Pipeline: Instruction Execution

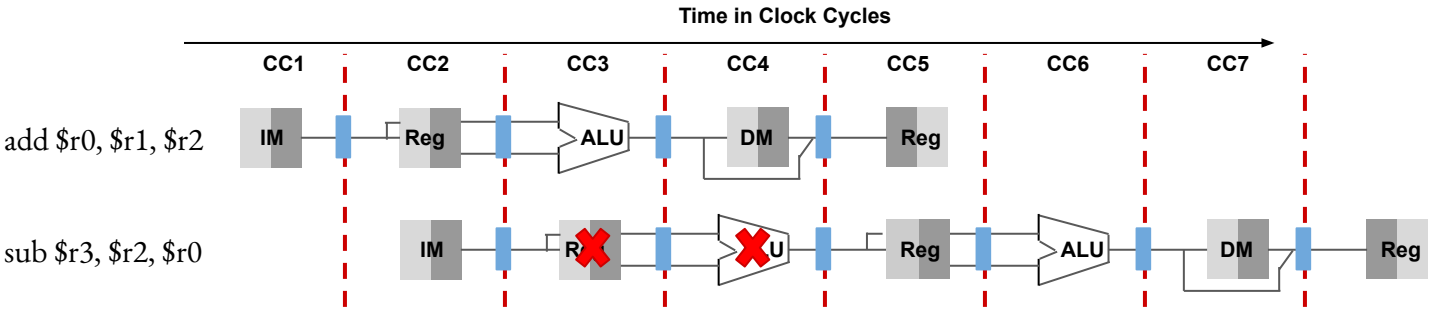


Pipeline Hazards

- **Situations in pipeline when the next instruction can’t execute in the following cycle**
- **Three types of hazards can take place-**
  - **Structural Hazard- planned instruction can not execute in proper clock cycle as the hardware does not support the combination of instructions that are set to execute**
  - **Data Hazard- execution stalls as data needed for execution is not yet ready/available**
  - **Control Hazard- an incorrect instruction has been fetched, which is not expected**

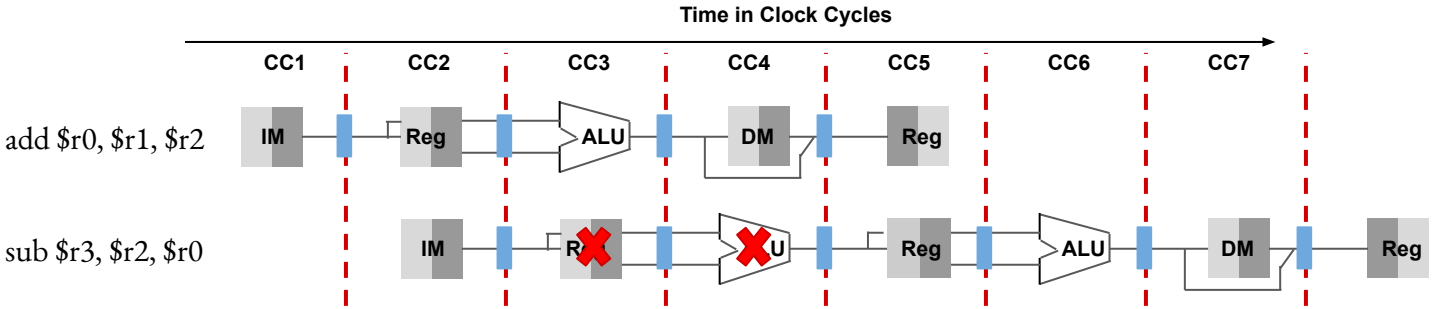
Data Hazards

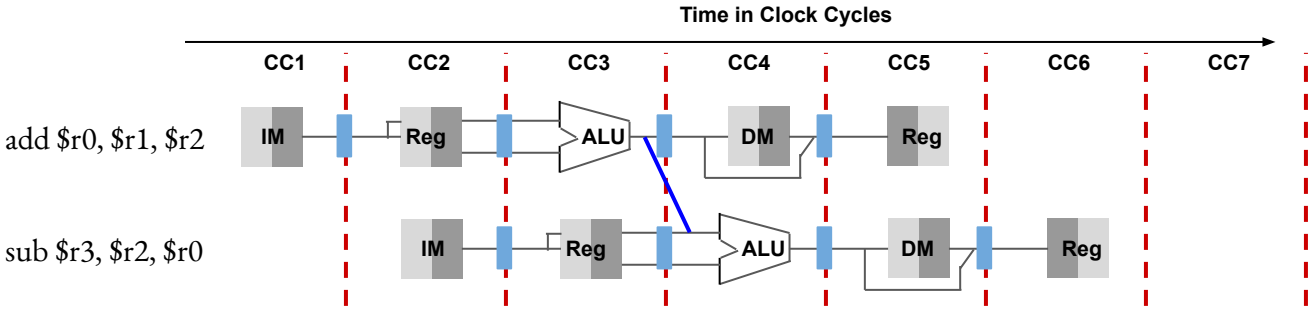
- **The Dependencies**
  - **Consider the following instructions-**
    - add $r0, $r1, $r2
    - sub $r3, $r2, $r0



- **Can not proceed due to stalls/bubbles**

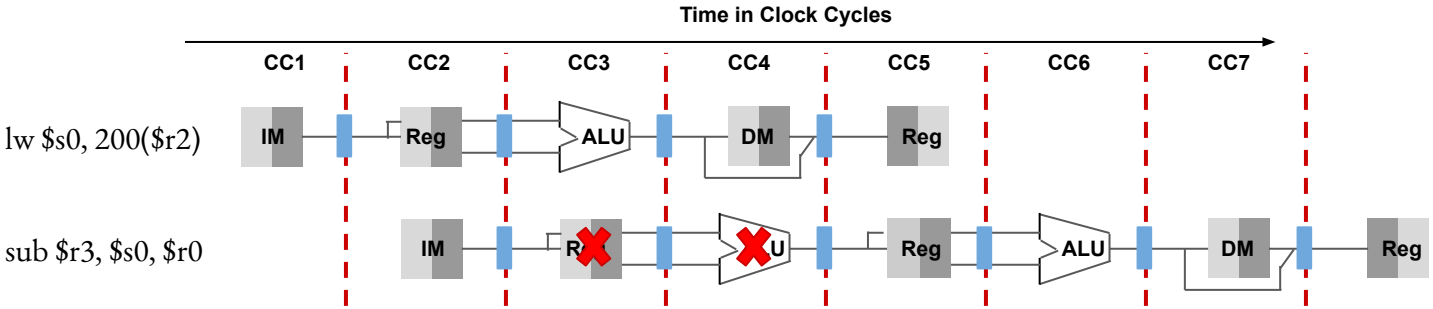
Data Hazards

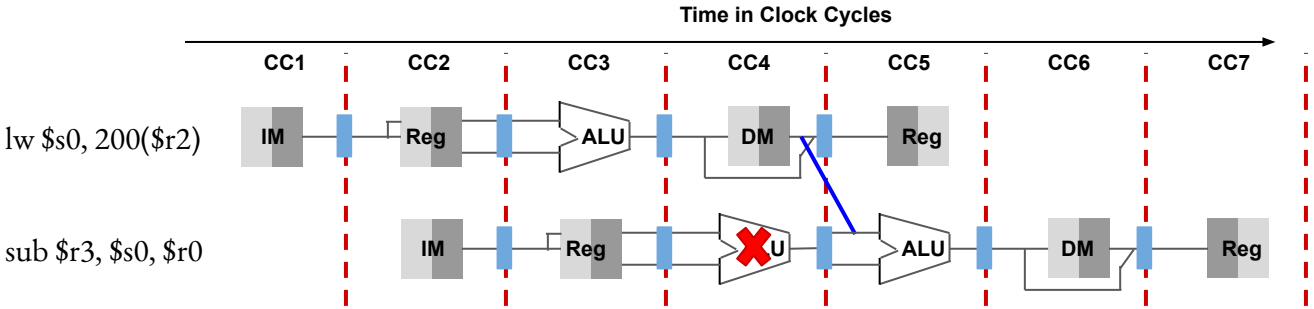




- **Forwarding**
  - **No stall**

Data Hazards

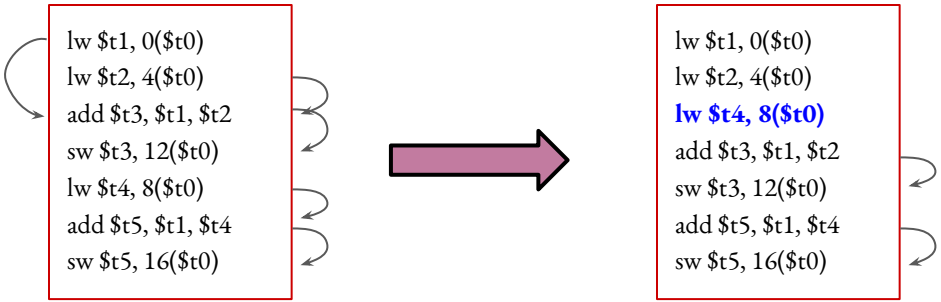




- **Forwarding**
  - **Still 1 stall**

Data Hazards

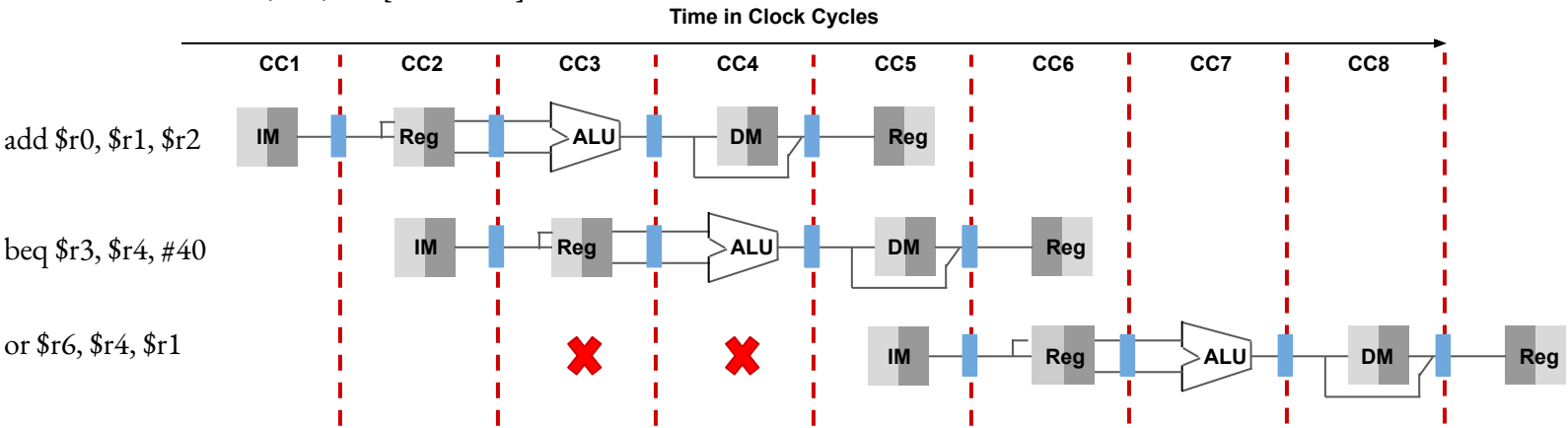
- **Can we shuffle instructions to avoid data hazards?**



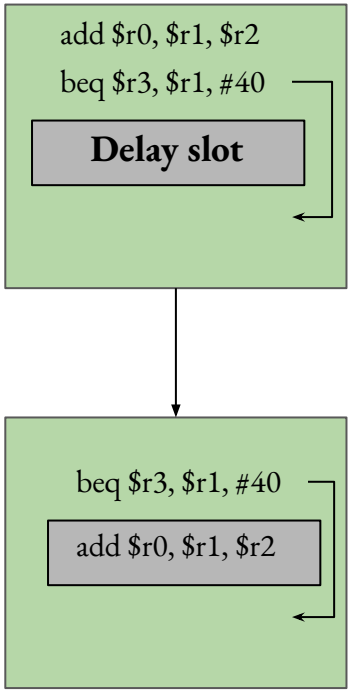
- **But who will help us with this?**
  - **Programmer?**
  - **Compilers?**
  - **Or, someone else?**

Control Hazards

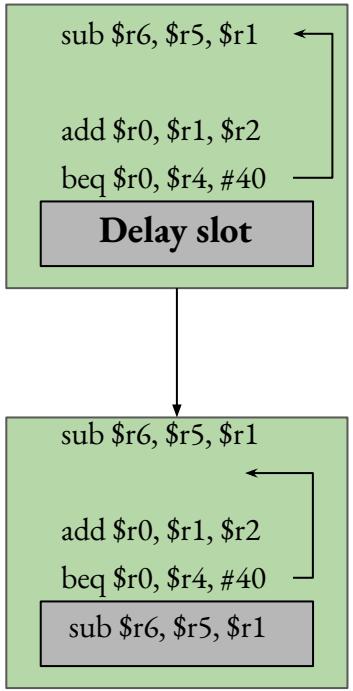
- **The Dependencies**
  - **Consider the following instructions-**
    - add $r0, $r1, $r2
    - beq $r3, $r4, #40
    - lw $r5, 200($r4)
    - or $r6, $r4, $r1 [Addr: #40]



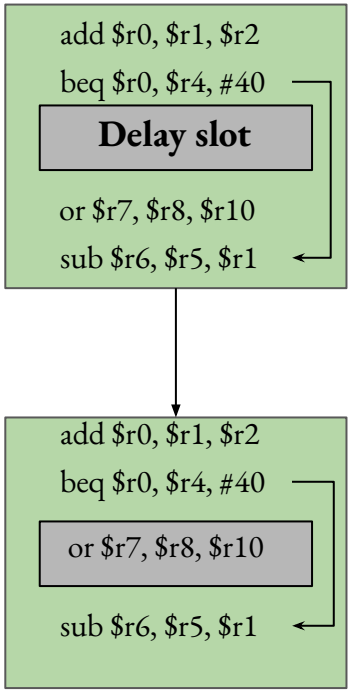
Control Hazards: Solutions??



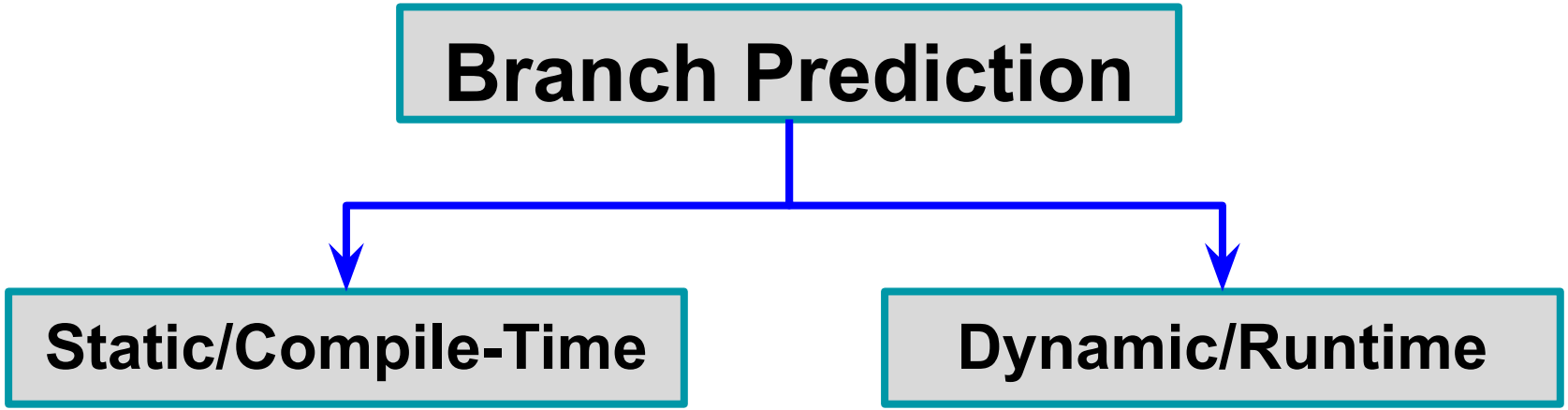
From Before



From Target



From Fall-Through


Branch Prediction
Static/Compile-Time
Dynamic/Runtime

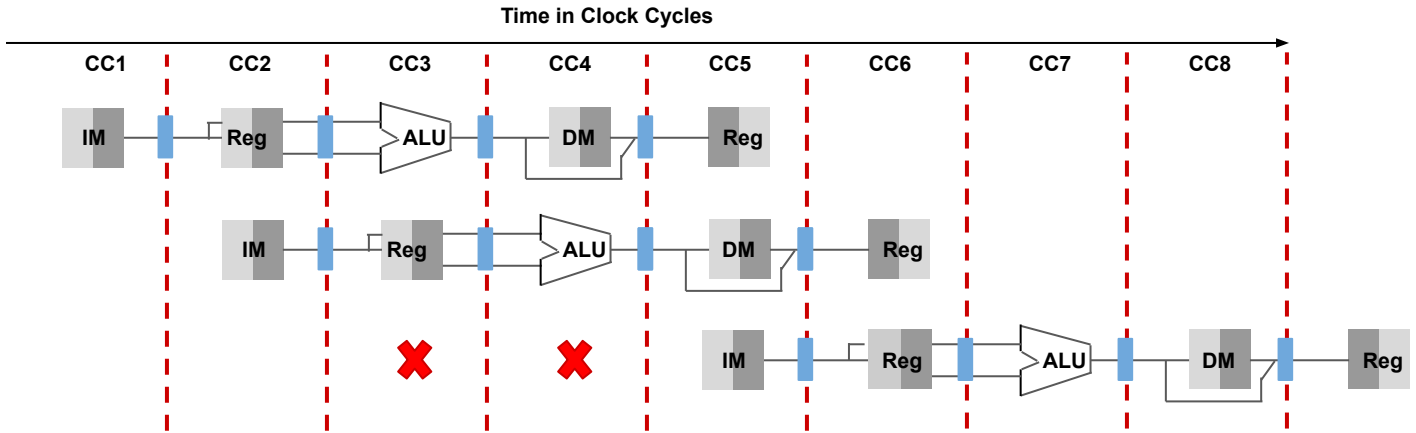
Dynamic Branch Prediction

Branches and Delays

```
a = 5;
b = 10;
if (a < b) {
    printf("5 is less than 10.");
} else {
    printf("5 is greater than 10.");
}
```

```
for (i = 1; i <= 5; i++) {
    printf("I am fine. \n");
}
```

```
while (i < 5) {
    printf("A sample loop. \n");
    i++;
}
```



Dynamic Branch Prediction

- **To predict branch outcome on-the-fly, a history table is required**
  - **Branch History Table (BHT): a small memory indexed by the lower portion of the address of the branch instruction along with a prediction bit**

- **A single bit predictor has lower implementation overhead**
  - **Is it really a good predictor?**

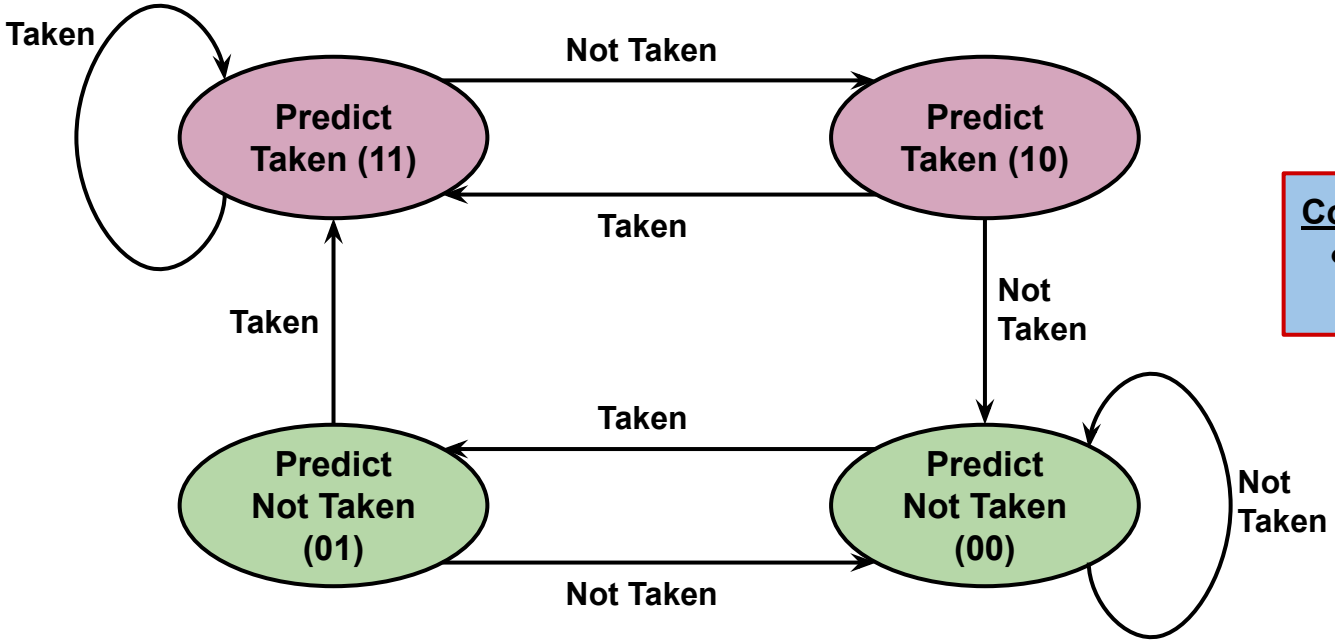
```
if (a < b) {
    printf("a is less than b.");
} else {
    printf("a is greater than or equal to
b.");
}
```

Case 1: 1111111100000000

Case 2: 1010101010101010

Dynamic Branch Prediction (Cont…)

- Can we have a 2-bit predictor?



Correlating Branch Predictors:

- consider dependencies between branches

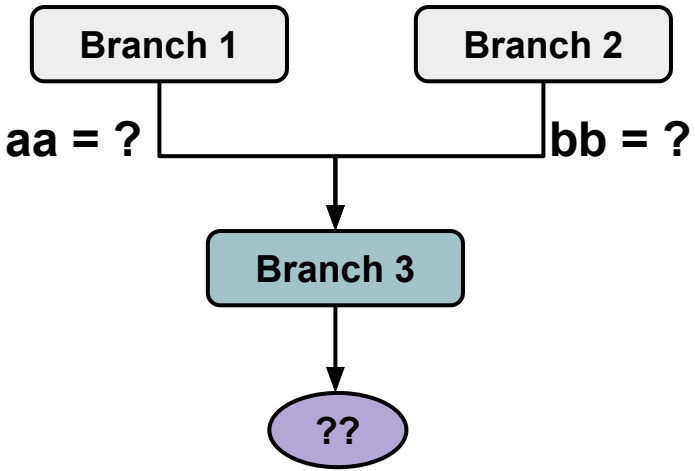
Correlating Branch Prediction

```
if (aa==2)
    aa=0;
if (bb==2)
    bb=0;
if (aa!=bb) {
    //…more codes…
}
```

Branch 1

Branch 2

Branch 3



Can we predict outcome of Branch 3?

Dynamic Scheduling

- Data Hazard (RAW):
  - `DIV.D F0,F2,F4`
  - `ADD.D F10,F0,F8`
  - `SUB.D F12,F8,F14`

- `ADD.D` depends on `DIV.D`; incurs stalls
- So, `SUB.D` will be delayed without having any dependencies

- Consider the code below:

```
DIV.D F0,F2,F4
ADD.D F6,F0,F8
SUB.D F8,F10,F14
MUL.D F6,F10,F8
```

- Possibilities of WAR and WAW Hazards
- What are the solutions?
  - Reordering instructions
  - Register Renaming

Out-of-Order Execution can solve the issue

Summary

- Single cycle datapath- the core architecture of processor
- Pipelined architecture to speed up
- Pipeline has its own limitations, called as hazards
  - Structural Hazard
  - Data Hazard
  - Control hazard
- Forwarding can reduce the issues related to Data Hazards
  - How to implement?
- Control Hazards can be managed by shuffling instructions and/or Dynamic Branch Prediction

Thank
You